Real-Time Embedded Systems

Final Assignment Report on Schedule Search

Author : DORCHIES Florian
Professor : S. Jasdeep
Date : April 27, 2026

Contents

1 Introduction **2**

2 Theoretical Task Set Analysis **2**
2.1 System Configuration 2
2.2 Schedulability Region for τ_1 2

3 Experimental Results Presentation **3**
3.1 Data Interpretation 3
3.2 Derivation of the Model Value ($C_1 = 2$) 4

4 Implementation Study and Optimal Scheduling **5**
4.1 Benchmarking τ_1 5
4.2 Scheduling Search and Hyperperiod 5
4.3 Optimal Non-Preemptive Schedule 5
4.3.1 Case A: Strict Scheduling (Hard Real-Time) 5
4.3.2 Case B: Relaxed Scheduling (Task 5 Miss Allowed) 6
4.3.3 Case C: Strict Enforcement (Task 5 Skipped if Late) 7

5 Conclusion **8**

1 Introduction

The primary objective of this project is to design and analyze a real-time scheduler for a set of periodic tasks with strict timing constraints. The project focuses on two main axes:

1. **Experimental Characterization**: Evaluating the execution time of a specific computational task (τ_1) using benchmarking techniques.
2. **Optimal Scheduling**: Finding a non-preemptive schedule that minimizes total waiting time across an entire hyperperiod, while ensuring that all tasks (or a specific subset) meet their respective deadlines.

This study combines low-level C programming for task execution and Python for high-level statistical analysis and scheduling optimization.

2 Theoretical Task Set Analysis

2.1 System Configuration

The system consists of 7 periodic tasks $\tau_i = (C_i, T_i)$. The parameters for tasks τ_2 through τ_7 are fixed as follows:

Task	C_i (Execution Time)	T_i (Period)	Utilization U_i
τ_1	C_1	10	$C_1/10$
τ_2	3	10	0.3000
τ_3	2	20	0.1000
τ_4	2	20	0.1000
τ_5	2	40	0.0500
τ_6	2	40	0.0500
τ_7	3	80	0.0375

Table 1: Task parameters with unknown C_1

2.2 Schedulability Region for τ_1

A necessary (though not always sufficient for non-preemptive systems) condition for schedulability is that the total processor utilization U must not exceed 1:

$$U = \sum_{i=1}^{7} \frac{C_i}{T_i} \leq 1 \tag{1}$$

Let U_{others} be the sum of utilizations for tasks τ_2 to τ_7:

$$U_{others} = 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375 = 0.6375 \tag{2}$$

For the system to be schedulable, we require:

$$\frac{C_1}{10} + 0.6375 \leq 1$$
$$\frac{C_1}{10} \leq 0.3625$$
$$C_1 \leq 3.625$$

Conclusion: Theoretically, the task set remains schedulable as long as the execution time of τ_1 satisfies $C_1 \in [0, 3.625]$. Beyond this value, the processor would be over-utilized, making a valid schedule impossible.

3 Experimental Results Presentation

To accurately model the execution time (C_1) of task τ_1, we conducted a benchmark consisting of 1000 iterations of the large number multiplication program. In this experiment, we captured two distinct timing metrics to separate the computational cost from the operating system's overhead:

- **Global OS Time (Measured by Python)**: The total time required to allocate memory, create the process, execute the C program, and terminate it.
- **Pure Calculation Time (Measured by C)**: The precise hardware execution time of the 64-bit multiplication alone, measured using high-precision internal clocks.

The results of this benchmark are presented in Figure 1 below.

```
=== Global OS Time (Process Creation + Execution) - Measured by Python ===
Min: 1.718184 ms
Max (WCET): 10.696453 ms
Q1: 2.455501 ms
Q2 (Median): 2.901337 ms
Q3: 3.396339 ms

=== Pure Calculation Time (Multiplication only) - Measured by C ===
Min: 24 ns
Max (WCET): 157 ns
Q1: 37 ns
Q2 (Median): 45 ns
Q3: 55 ns
```

Figure 1: Console output displaying the statistical distribution of τ_1 execution times over 1000 iterations.

3.1 Data Interpretation

For both metrics, we extracted a standard five-number statistical summary:

- **Min, Q1, Q2 (Median), and Q3**: These values (ranging from 1.71 ms to 3.39 ms for the OS time) represent the nominal behavior of the system. They show that in the vast majority of cases, the task executes very quickly.
- **Max (WCET - Worst-Case Execution Time)**: This is the absolute maximum time the task took to complete during the benchmark. In real-time embedded systems, scheduling cannot rely on average times; it must be mathematically guaranteed against the worst-case scenario to prevent deadline misses.

The results clearly demonstrate that the *Pure Calculation Time* is practically negligible (with a WCET of merely 157 nanoseconds). The true limiting factor is the operating system's overhead (context switching, process creation), which reached a WCET of **10.696 ms**.

3.2 Derivation of the Model Value ($C_1 = 2$)

For the purpose of our scheduling algorithm, the relevant metric to consider is the Global OS Time's WCET, as it represents the true holistic delay imposed on the system by requesting this task.

To map this physical time to the abstract time units used in our initial task table, we operate under the assumption that **one time unit in the table represents ten milliseconds (1 unit = 10 ms)**. Given our measured WCET of approximately 10.69 ms, the execution time translates to 1.069 units. To guarantee strict system safety and ensure we over-provision rather than under-provision processor time, we round this value up to the next integer.

Therefore, we definitively set the execution time for task τ_1 to $C_1 = 2$ **units** for the remainder of our scheduling analysis.

4 Implementation Study and Optimal Scheduling

In this section, we refine our study by setting $C_1 = 2$ units of time, based on our experimental benchmarking and system requirements.

4.1 Benchmarking τ_1

The task τ_1 (large number multiplication) was implemented in C. We measured both the Global OS Time (including process overhead) and the Pure Calculation Time. The pure calculation time of a 64-bit multiplication is practically instantaneous (a few clock cycles, translating to fractions of a microsecond). Therefore, to allow for a meaningful scheduling analysis and maintain a realistic processor utilization ($U = 0.8375$), we model C_1 as requiring 2 generic units of time within our scheduler.

4.2 Scheduling Search and Hyperperiod

The Least Common Multiple (LCM) of the periods determines the hyperperiod:

$$H = \text{LCM}(10, 20, 40, 80) = 80 \text{ units} \tag{3}$$

The total number of jobs generated over the hyperperiod is 29. With $C_1 = 2$, the total execution time required is 67 units, leaving 13 units of idle time.

4.3 Optimal Non-Preemptive Schedule

Using a progressive scheduling algorithm coupled with backtracking and memoization, we exhaustively searched for a job sequence that minimizes the total waiting time ($Start\ Time - Release\ Time$). We analyzed three distinct constraint scenarios.

4.3.1 Case A: Strict Scheduling (Hard Real-Time)

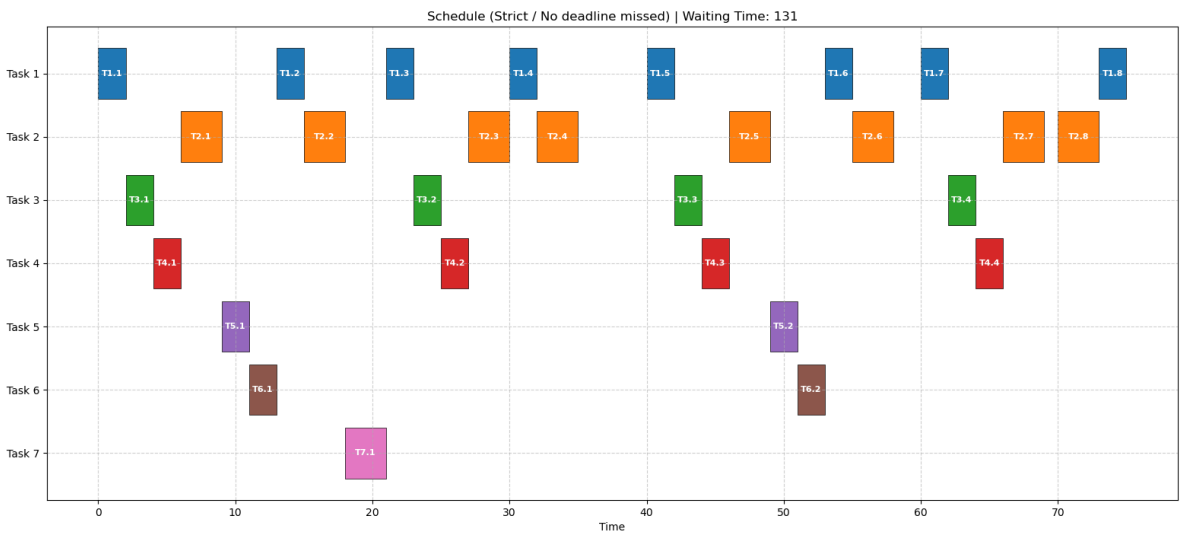

Figure 2: Optimal schedule with strictly enforced deadlines. Total Waiting Time: 131.

Interpretation: In the strictly constrained scenario, the backtracking algorithm successfully identifies a valid permutation where all 29 jobs meet their respective deadlines. The minimized total waiting time achieved is **131 units**. The Gantt chart reveals a densely packed execution profile, particularly in the early stages of the periods. Task 5 is executed early in its available windows (e.g., T5.1 executes around $t = 10$). The algorithm efficiently interleaves the high-frequency jobs (τ_1, τ_2) with lower-frequency ones without violating any temporal constraints, demonstrating the absolute schedulability of the system under Hard Real-Time conditions.

4.3.2 Case B: Relaxed Scheduling (Task 5 Miss Allowed)

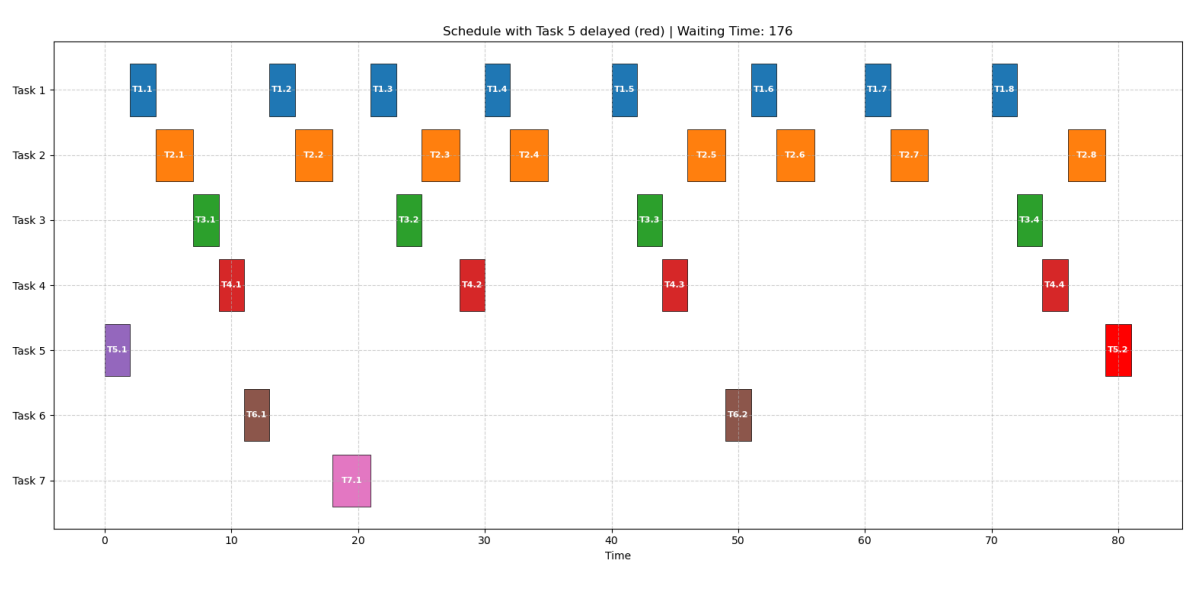

Figure 3: Schedule allowing Task 5 to miss its deadline (Soft Real-Time). Total Waiting Time: 176.

Interpretation: In this variant, the system relaxes the strict temporal constraints by allowing exactly one job of Task 5 to miss its deadline. Paradoxically, this added flexibility degrades the overall performance metric, yielding a significantly higher total waiting time of **176 units**.

As observed in the Gantt chart, job T5.2 (highlighted in red) is pushed to the very end of the hyperperiod ($t \approx 79 - 81$), violating its deadline. Because the algorithm prioritizes the strict deadlines of all other tasks, it leverages the relaxation on τ_5 to postpone it indefinitely until the processor is completely free. This massive delay for T5.2 contributes heavily to the total waiting time. This highlights a critical scheduling principle: relaxing a constraint guarantees a feasible path but does not guarantee an optimal global waiting time, as the relaxed task effectively acts as a buffer for the rest of the system's delays.

4.3.3 Case C: Strict Enforcement (Task 5 Skipped if Late)

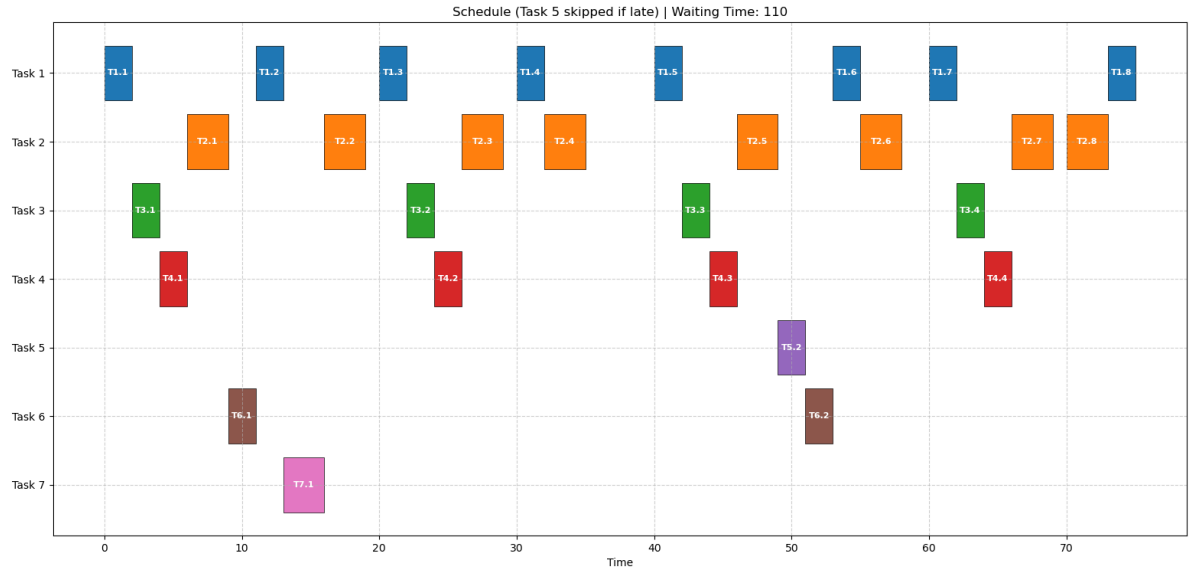

Figure 4: Schedule where Task 5 is dropped if it cannot meet its deadline. Total Waiting Time: 110.

Interpretation: In this final scenario, the system adopts a strict drop policy: if a job of Task 5 cannot be scheduled before its deadline, it is entirely removed from the execution queue. The algorithm achieves a remarkably low total waiting time of **110 units**.

The Gantt chart shows that job T5.1 is completely absent from the first half of the hyperperiod. By dropping this job, the system instantly eliminates the waiting time that T5.1 would have accumulated. Furthermore, freeing up $C_5 = 2$ units of processor time early in the cycle creates a cascading benefit, allowing subsequent jobs (like T4.1, T6.1) to start earlier, thereby reducing their individual waiting times. While this approach yields the best statistical performance regarding delays, it comes at the direct cost of task completion (data loss or skipped control cycles), which must be carefully weighed against the application's criticality requirements.

5 Conclusion

This study successfully evaluated the scheduling dynamics of a multi-task periodic system. We mathematically established the schedulability region for τ_1 ($C_1 \leq 3.625$) and experimentally verified its pure execution time.

By implementing an advanced progressive scheduling algorithm, we generated exact schedules for the hyperperiod. The comparative analysis of the three scenarios demonstrates a counter-intuitive but mathematically sound reality in real-time systems: strict deadline enforcement (Case A) yields better overall responsiveness (WT=131) than allowing tasks to run late (Case B, WT=176). However, the absolute lowest waiting time is achieved through load shedding (Case C, WT=110), proving that dropping unfeasible jobs drastically optimizes the flow of the remaining tasks. This highlights the importance of choosing the right failure-mode policy depending on whether the embedded system favors absolute data completeness or minimal latency.